

threshold value t and add noise. The exact behavior of the method depends on the type of noise ξ and the choice of μ and σ , which we can pick as functions of x in order to let some gradients be propagated even when we are in the saturating regime.

A desirable property we would like ϕ to approximately satisfy is that, in expectation, it is equal to the hard-saturating activation function, i.e.

$$E_{\xi \sim \mathcal{N}(0,1)}[\phi(x, \xi)] \approx h(x) \quad (7)$$

If the ξ distribution has zero mean then this property can be satisfied by setting $\mu = 0$, but for biased noise it will be necessary to make other choices for μ . In practice, we used slightly biased ϕ with good results.

Intuitively we would like to add more noise when x is far into the saturated regime, since a large change in parameters would be required to desaturate h . Conversely, when x is close to the saturation threshold a small change in parameters would be sufficient for it to escape. To that end we make use of the difference between the original activation function h and its linearization u

$$\Delta = h(x) - u(x) \quad (8)$$

when choosing the scale of the noise. See Eqs.1 for definitions of u for the hard-sigmoid and hard-tanh respectively. The quantity Δ is zero in the unsaturated regime, and when h saturates it grows proportionally to the distance between $|x|$ and the saturation threshold x_t . We also refer $|\Delta|$ as the magnitude of the saturation.

We experimented with different ways of scaling σ with Δ , and empirically found that the following formulation performs better:

$$\begin{aligned} \sigma(x) &= c (g(p\Delta) - 0.5)^2 \\ g(x) &= \text{sigmoid}(x). \end{aligned} \quad (9)$$

In Equation 9 a free scalar parameter p is learned during the course of training. By changing p , the model is able to adjust the magnitude of the noise and that also effects the sign of the gradient as well. The hyper-parameter c changes the scale of the standard deviation of the noise.

4.1. Derivatives in the Saturated Regime

In the simplest case of our method we draw ξ from an unbiased distribution, such as a standard normal. In this case we choose $\mu = 0$ to satisfy Equation 7 and therefore we will have,

$$\phi(x, \xi) = h(x) + \sigma(x)\xi$$

Due to our parameterization of $\sigma(x)$, when $|x| \leq x_t$ our stochastic activation function behaves exactly as the linear function $u(x)$, leading to familiar territory. Because Δ will be 0. Let us for the moment restrict our attention to the case

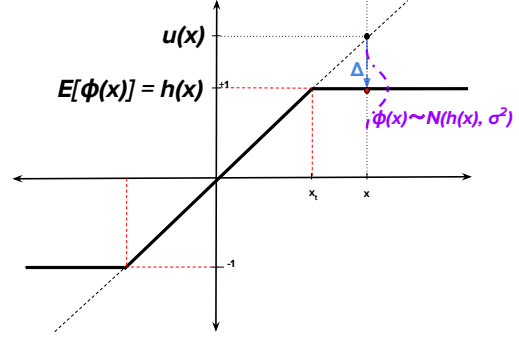


Figure 3. A simple depiction of adding Gaussian noise on the linearized activation function, which brings the average back to the hard-saturating nonlinearity $h(x)$, in bold. Its linearization is $u(x)$ and the noisy activation is ϕ . The difference $h(x) - u(x)$ is Δ which is a vector indicates the discrepancy between the linearized function and the actual function that the noise is being added to $h(x)$. Note that, Δ will be zero, at the non-saturating parts of the function where $u(x)$ and $h(u)$ matches perfectly.

when $|x| > x_t$ and h saturates. In this case the derivative of $h(x)$ is precisely zero, however, if we condition on the sample ξ we have

$$\phi'(x, \xi) = \frac{\partial}{\partial x} \phi(x, \xi) = \sigma'(x)\xi \quad (10)$$

which is non-zero almost surely.

In the non-saturated regime, where $\phi'(x, \xi) = h'(x)$ the optimization can exploit the linear structure of h near the origin in order to tune its output. In the saturated regime the randomness in ξ drives exploration, and gradients still flow back to x since the scale of the noise still depends on x . To reiterate, we get gradient information at every point in spite of the saturation of h , and the variance of the gradient information in the saturated regime depends on the variance of $\sigma'(x)\xi$.

4.2. Pushing Activations towards Linear Regime

An unsatisfying aspect of the formulation with unbiased noise is that, depending on the value of ξ occasionally the gradient of ϕ will point the wrong way. This can cause a backwards message that would push x in a direction that would worsen the objective function on average over ξ . Intuitively we would prefer these messages to “push back” the saturated unit towards a non-saturated state where the gradient of $h(x)$ can be used safely.

A simple way to achieve this is to make sure that the noise ξ is always positive and adjust its sign manually to match the sign of x . In particular we could set

$$\begin{aligned} d(x) &= -\text{sgn}(x) \text{sgn}(1 - \alpha) \\ s &= \mu(x) + d(x)\sigma(x)|\xi|. \end{aligned}$$

where ξ and σ are as before and sgn is the sign function, such that $\text{sgn}(x)$ is 1 if x is greater than or equal to 0 otherwise it is

–1. We also use the absolute value of ξ in the reparametrization of the noise, such that the noise is being sampled from a half-Normal distribution. We ignored the sign of ξ , such that the direction that the noise pushes the activations are determined by $d(x)$ and it will point towards $h(x)$. Matching the sign of the noise to the sign of x would ensure that we avoid the sign cancellation between the noise and the gradient message from backpropagation. $\text{sgn}(1-\alpha)$ is required to push the activations towards $h(x)$ when the bias from α is introduced.

In practice we use a hyperparameter α that influences the mean of the added term, such that α near 1 approximately satisfies the above condition, as seen in Fig. 4. We can rewrite the noisy term s in a way that the noise can either be added to the linearized function or $h(x)$. The relationship between Δ , $u(x)$ and $h(x)$ is visualized Figure 4.1 can be expressed as in Eqn 11.

We have experimented with different types of noise. Empirically, in terms of performance we found, half-normal and normal noise to be better. In Eqn 11, we provide the formulation for the activation function where $\epsilon = |\xi|$ if the noise is sampled from half-normal distribution, $\epsilon = \xi$ if the noise is sampled from normal distribution.

$$\phi(x, \xi) = u(x) + \alpha\Delta + d(x)\sigma(x)\epsilon \quad (11)$$

By using Eqn 11, we arrive at the noisy activations, which we used in our experiments.

$$\phi(x, \xi) = \alpha h(x) + (1 - \alpha)u(x) + d(x)\sigma(x)\epsilon \quad (12)$$

As can be seen in Eqn 12, there are three paths that gradients can flow through the neural network, the linear path ($u(x)$), nonlinear path ($h(x)$) and the stochastic path ($\sigma(x)$). The flow of gradients through these different pathways across different layers makes the optimization of our activation function easier.

At test time, we used the expectation of Eqn 12 in order to get deterministic units,

$$\mathbb{E}_\xi[\phi(x, \xi)] = \alpha h(x) + (1 - \alpha)u(x) + d(x)\sigma(x)\mathbb{E}_\xi[\epsilon] \quad (13)$$

If $\epsilon = \xi$, then $\mathbb{E}_\xi[\epsilon]$ is 0. Otherwise if $\epsilon = |\xi|$, then $\mathbb{E}_\xi[\epsilon]$ is $\sqrt{\frac{2}{\pi}}$.

Algorithm 1 Noisy Activations with Half-Normal Noise for Hard-Saturating Functions

- 1: $\Delta \leftarrow h(x) - u(x)$
- 2: $d(x) \leftarrow -\text{sgn}(x) \text{sgn}(1 - \alpha)$
- 3: $\sigma(x) \leftarrow c(g(p\Delta) - 0.5)^2$
- 4: $\xi \sim \mathcal{N}(0, 1)$
- 5: $\phi(x, \xi) \leftarrow \alpha h(x) + (1 - \alpha)u(x) + (d(x)\sigma(x)|\xi|)$

To illustrate the effect of α and noisy activation of the hard-tanh, We provide plots of our stochastic activation functions in Fig 4.

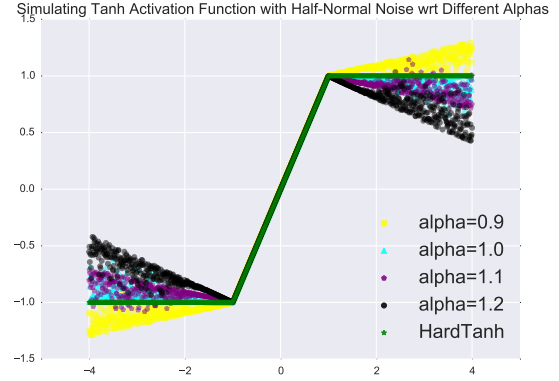


Figure 4. Stochastic behavior of the proposed noisy activation function with different α values and with noise sampled from the Normal distribution, approximating the hard-tanh nonlinearity (in bold green).

5. Adding Noise to Input of the Function

Adding noise with fixed standard deviation to the input of the activation function has been investigated for ReLU activation functions (Nair & Hinton, 2010; Bengio et al., 2013).

$$\phi(x, \xi) = h(x + \sigma\xi) \text{ and } \xi \sim \mathcal{N}(0, 1). \quad (14)$$

In Eqn 14, we provide a parametrization of the noisy activation function. σ can be either learned as in Eqn 9 or fixed as a hyperparameter.

The condition in Eqn 5 is satisfied only when σ is learned. Experimentally we found small values of σ to work better. When σ is fixed and small, as x gets larger and further away from the threshold x_t , noise will less likely be able to push the activations back to the linear regime. We also investigated the effect of injecting input noise when the activations saturate:

$$\phi(x, \xi) = h(x + \mathbf{1}_{|x| \geq |x_t|}(\sigma\xi)) \text{ and } \xi \sim \mathcal{N}(0, 1). \quad (15)$$

6. Experimental Results

In our experiments, we used noise only during training: at test time we replaced the noise variable with its expected value. We performed our experiments with just a drop-in replacement of the activation functions in existing experimental setups, without changing the previously set hyper-parameters. Hence it is plausible one could obtain better results by performing a careful hyper-parameter tuning for the models with noisy activation functions. In all our experiments, we initialized p uniform randomly from the range $[-1, 1]$.

We provide experimental results using noisy activations with normal (NAN), half-normal noise (NAH), normal noise at the input of the function (NANI), normal noise at the input of the function with learned σ (NANIL) and normal noise

injected to the input of the function when the unit saturates (NANIS). Codes for different types of noisy activation functions can be found at <https://xx>.

6.1. Exploratory Analysis

As a sanity-check, we performed small-scale control experiments, in order to observe the behavior of the noisy units. In Fig 5, we showed the learning curves of different types of activations with various types of noise in contrast to the tanh and hard-tanh units. The models are single-layer MLPs trained on MNIST for classification and we show the average negative log-likelihood $-\log P(\text{correct class}|\text{input})$. In general, we found that models with noisy activations converge faster than those using tanh and hard-tanh activation functions, and to lower NLL than the tanh network.

We trained 3-layer MLP on a dataset generated from a mixture of 3 Gaussian distributions with different means and standard deviations. Each layer of the MLP contains 8-hidden units. Both the model with tanh and noisy-tanh activations was able to solve this task almost perfectly. By using the learned p values, in Figure 6 and 7, we showed the scatter plot of the activations of each unit at each layer and the derivative function of each unit at each layer with respect to its input.

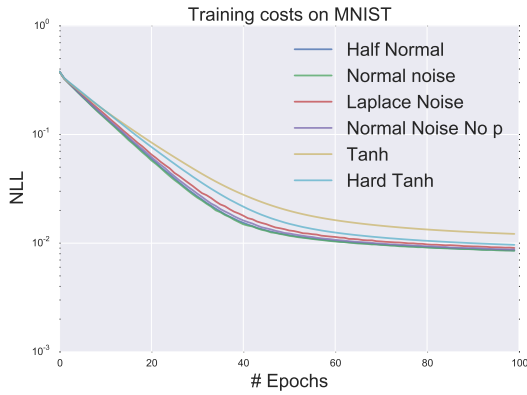


Figure 5. Learning curves of a single layer MLP trained with RMSProp with different noise types and activation functions

We further investigated the performances of network with activation functions using NAN, NANI and NANIS on penn-treebank (PTB) character-level language modeling. We used a GRU language model over sequences of length 200. We used the same model and train all the activation functions with the same hyperparameters except we ran a grid-search for σ for NANI and NANIS from $[1, 0.01]$ with 8 values. We choose the best σ based on the validation bit-per-character (BPC). We have not observed important difference among NAN and NANI in terms of training performance as seen on Figure 8.

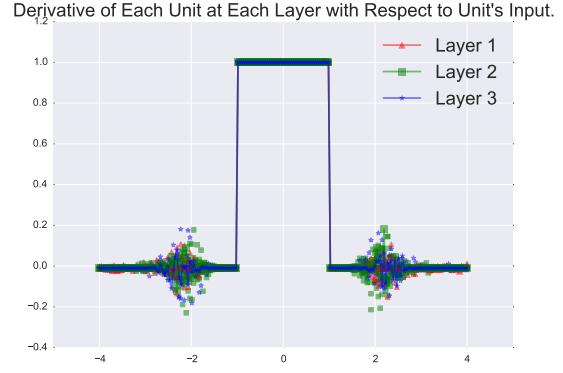


Figure 6. Derivatives of each unit at each layer with respect to its input for a three-layered MLP trained on a dataset generated by three normal distributions with different means and standard deviations. In other words learned $\frac{\partial \phi(x_i^k, \xi_i^k)}{\partial x_i^k}$ at the end of training for i^{th} unit at k^{th} layer. ξ^k is sampled from Normal distribution with $\alpha = 1$.

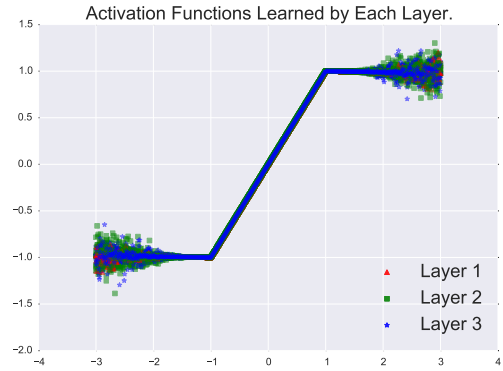


Figure 7. Activations of each unit at each layer of a three-layer MLP trained on a dataset generated by three normal distributions with different means and standard deviations. In other words learned $\phi(x_i^k, \xi_i^k)$ at the end of training for i^{th} unit at k^{th} layer. ξ^k is sampled from Half-Normal distribution with $\alpha = 1$.

6.2. Learning to Execute

The problem of predicting the output of a short program introduced in (Zaremba & Sutskever, 2014)¹ proved challenging for modern deep learning architectures. The authors had to use curriculum learning (Bengio et al., 2009) to let the model capture knowledge about the easier examples first and increase the level of difficulty of the examples further down the training.

We replaced all sigmoid and tanh non-linearities in the reference model with their noisy counterparts. We changed

¹The code is residing at https://github.com/wojciechz/learning_to_execute. We thank authors for making it publicly available.